Task 3 writeup

Mike Neuder, COS 568 Project, Spring 2026

My hybrid implementation is in `hybrid_pgm_lipp.h`. I ran many experiments with more complicated setups, but settled on a relatively simple one. I kept the complex version in `hybrid_complex.h` and will discuss that approach and my hypothesis for why it was unable to perform as well as the simpler one under the header (Attempts at a more complex solution) after sharing my main results first. Plots in `analysis-task3.ipynb` and pull data from `results*` directories (`results12/` is the main run I discuss in the writeup).

Main results: relatively-simple solution

Overview

The core components of my hybrid approach are:

1. **LIPP Index:** This is the index that the original keys are loaded into.
2. **DPGM Index:** The DPGM index is initially empty and all newly inserted keys are added to it.
3. **Bloom filter:** The bloom filter is used to check whether a key is in the DPGM index.

The bloom filter is extremely important on the lookup flow, because 50% of the lookups are negative (meaning the key isn't in the dataset). As such, the bloom filter can allow us to return `NOT_FOUND` faster than performing the slow lookup in the DPGM index. Even though the bloom filter is probabilistic, it *never has false negatives*. That is, it may sometimes return that the key is in the index when it really isn't, but it will never return that the key is not in the index when it really is. This preserves the correctness of our negative lookups.

Lookup flow

Thus, the full lookup flow is as follows:

1. **Check LIPP index:** Since LIPP is lookup optimized, this happens extremely quickly and in the happy path, it will return the data iMediately. We call the function directly on the LIPP index: `lipp_.find(lookup_key, value)`.
2. **Check the bloom filter:** If the LIPP index returns `NOT_FOUND`, then we query the bloom

filter for the key with `bloom_.maybe_contains(lookup_key)`. The function is called `maybe_contains` because this is a non-deterministic data structure, meaning it might report a false positive. If the bloom filter returns `NOT_FOUND`, we know that this is a negative lookup and can return iMediately. If the bloom filter returns `true`, then we have to go to the actual DPGM index to retrieve the value corresponding to the lookup key.

3. **Check the DPGM index:** This is the slow path. We have to use the iterator `pgm_buffer_.find(lookup_key)` to check the DPGM index for the key.

Insert flow

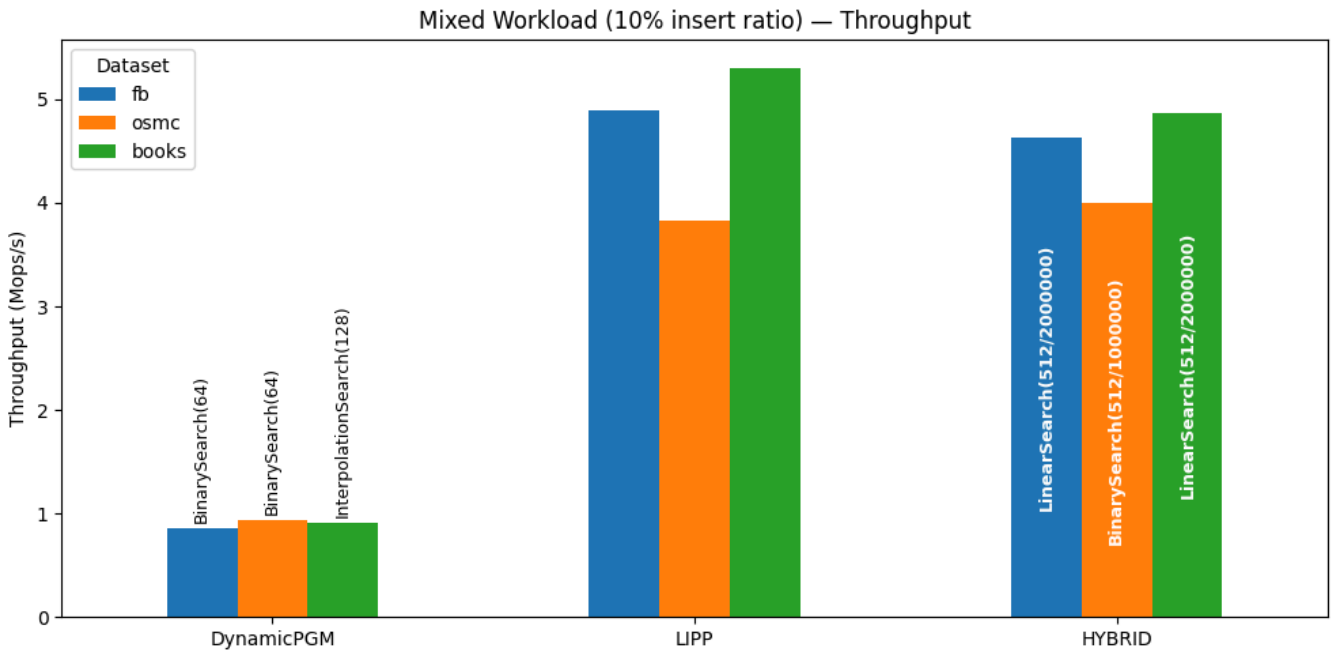
The insert flow is as follows:

1. **Insert into the DPGM index:** Since we are taking advantage of the fast inserts of DPGM, we just go straight into that index with `pgm_buffer_.insert(data.key, data.value)`.
2. **Insert into the bloom filter:** We also have to insert this key into the bloom filter, because any lookup needs to return true. We add it with `bloom_.insert(data.key)`.
3. **(Potentially) flush:** The flush mechanism is extremely simple. It just manually inserts all of the values from the DPGM index into the LIPP index one by one. (The more complex solution outlined below does fancier things.)

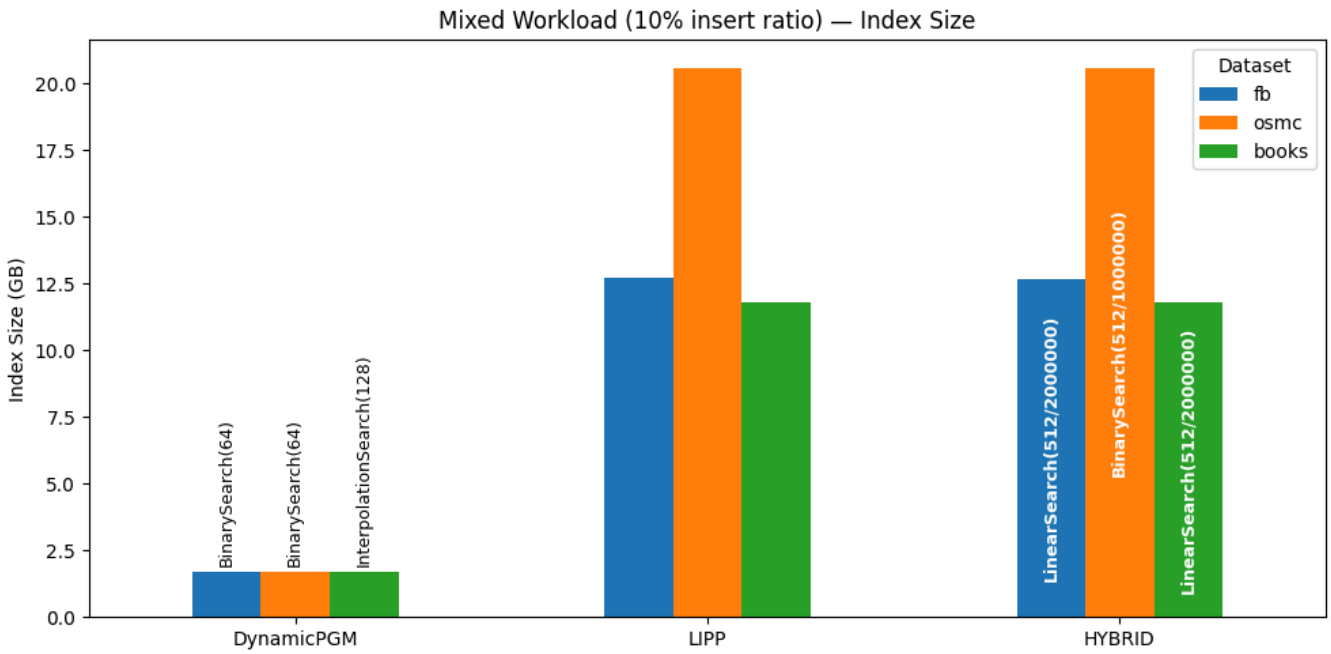
Results

The bar plots show my last run (data in `results12/` directory). All runs were done on Adroit using the `task3.slurm` file, which runs each experiment three times with `-r 3`.

Starting with the 10% inserts workload:



- Overall, the hybrid index performs about as well as LIPP. It was very hard to beat the LIPP performance on this benchmark, because with only 10% inserts, the lookup optimization of LIPP is strong. The hybrid index outperformed LIPP on OSMC, but only by a few percentage points so that is probably just inter-run noise.
- Across the board, both LIPP and hybrid meaningfully outperform DPGM on this workload. This makes sense because most of the lookups are being processed by the LIPP index.
- I tried various sizes for the `flush_threshold`, which determines when to flush based on the size of the DPGM index. In this workload, there are 2 million operations, and with 10% of them being inserts, that means there will be 200k inserts. The bar plot labels show that the best results were with very big flush thresholds, meaning there will be no flushing.

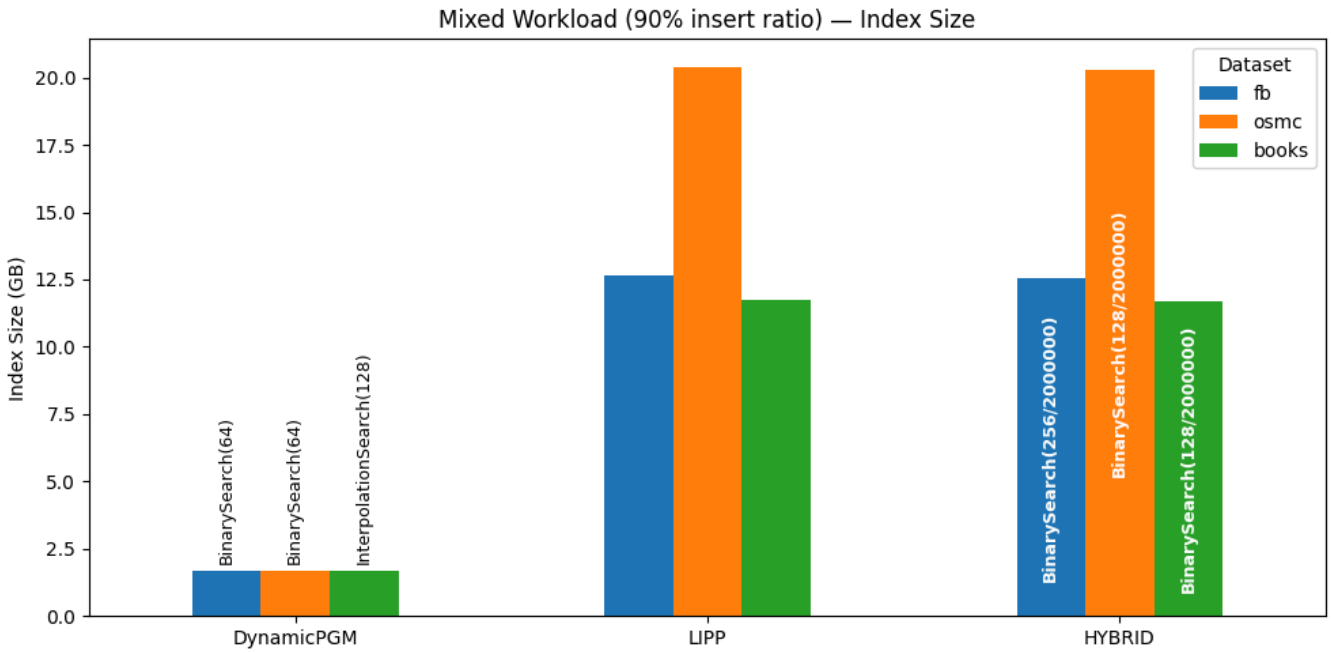


- The LIPP and hybrid approaches have much bigger index sizes than DPGM. But between the two, they are extremely similar.

Now to the 90% inserts workload:

- Here we see major improvements of the hybrid benchmark over the others. In particular, we see about 40% increase in throughput across all of the tasks.
- In all three of these cases, the 2M flush threshold was best, meaning the larger DPGM indexes are good. With 90% of the 2M operations being inserts (1.8M inserts), setting the

flush threshold to 2M is the only version that doesn't result in a flush ever, which was important for the performance.



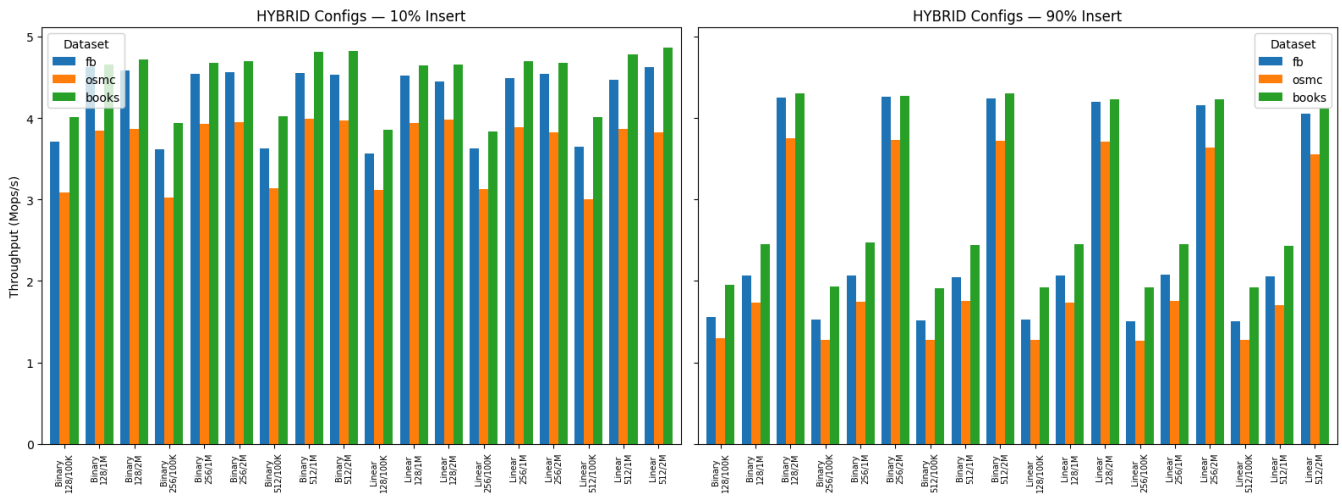
- Again, The LIPP and hybrid approaches have much bigger index sizes than DPGM. But between the two, they are extremely similar.

Hyperparameters tried

I swept 18 different configurations for each run:

- DPGM search method: `BranchingBinarySearch`, `LinearSearch`
- DPGM error: 128, 256, 512
- Flush threshold: 100k, 1M, 2M

The following plot shows the throughputs of the different configurations.



- We see that for the 10% insert workload, the size of the flush threshold between 1M and 2M didn't change things, since in either case it was never flushing (whereas the 100k flush threshold fires twice).
- For the 90% insert workload, the 2M flush threshold far outperformed the others by avoiding the flushing.

Attempts at a more complex solution

I tried a variety of more complex, doubled-buffered strategies, but was never able to find a good set of hyper-parameters that utilized the complexity to good effect. I will outline one, more complex approach that I left in `hybrid_complex.h` to demonstrate some of the things I tried.

Overview

The core components of the more advanced approach were

1. **Primary LIPP index:** The main lookup index that the initial data is bulk loaded into.
2. **Secondary LIPP index:** A backup LIPP index that is constructed from scratch when a flush occurs.
3. **Primary DPGM index:** The main insert index that loads any initial inserts.
4. **Secondary DPGM index:** A secondary insert index that handles inserts after a flush.
5. **Primary Bloom filter:** A bloom filter to check for key presence in the primary DPGM index.
6. **Secondary Bloom filter:** A bloom filter to check for key presence in the secondary DPGM index.

The key idea here was to try to make the flush mechanism more efficient by:

- Constructing a brand new LIPP index instead of inserting each key manually into the old one.
- Using a secondary DPGM index to minimize the impact of lookups after the first one hits the flush threshold.

Lookup flow

Thus, the full lookup flow was as follows:

1. **Check primary LIPP index:** This is the lookup optimized and has the full set of bulk loaded initial keys.
2. **If not flushed, check the bloom filter to check for key presence in the primary DPGM index:** This is the same flow as the simpler solution described above. Flushed is a bool that indicates whether or not a flush has occurred yet. If the bloom check returns true, then the lookup on the primary DPGM index takes place.
3. **If flushed, check the secondary LIPP index:** If flushed is true, then the primary DPGM index was used to construct a secondary LIPP index. Thus the key could be in there. This second lookup should be pretty quick too because of the speed of LIPP lookups.
4. **If flushed but not in the secondary LIPP index, check the secondary bloom filter:** Just as before, if flushed has occurred, then we know that some inserts could have been pushed into the secondary DPGM index. We use a second bloom filter to avoid slow negative lookups on the secondary DPGM index.

On net, the lookup flow was trying to exploit the "single flush" mechanic and amortize slow lookups over smaller DPGM indexes and amortize the slow flush into a bulk construction of a new LIPP index.

Insert flow

The insert flow is as follows:

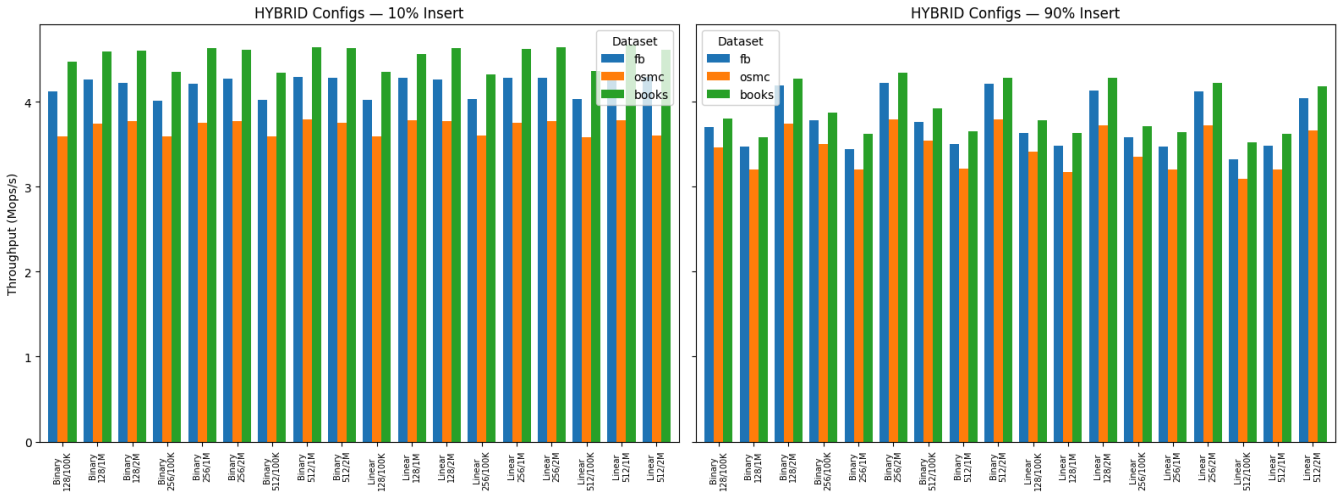
1. **If not flushed, insert into the primary DPGM index:** Since we are taking advantage of the fast inserts of DPGM, we just go straight into that index.
2. **If not flushed, insert into the primary bloom filter:** We also have to insert this key into the bloom filter.
3. **(Potentially) flush:** This flush mechanism is a little more complex. It sees if we have reached the flush threshold. If so, we grab all the data from the primary DPGM index and

sort the keys before bulk loading it into a secondary LIPP index. This is faster than inserting each key one-by-one.

4. **After the flush:** Once the flush is done, the new inserts go into the secondary DPGM index (and the corresponding secondary bloom filter) and any new lookups have to do the double LIPP check.

Results

The figure below shows the various throughputs based on this more complex solution.



- The results here are much more homogeneous than the simple solution. Basically, this tells us that the complexity helps in the worst-case, but doesn't improve the best-case much. The simplicity of only having a single LIPP, DPGM, and bloom filter are beneficial in the large flush threshold regimes, eliminating the need for the complexity introduced in this solution.
- Still, it is interesting to see how similar the various hyper-parameters perform under the more complex solution.

I also made attempts to use concurrency for the flush mechanism, but never found a solution that benefited much from this. My theory is that the thread safety of such techniques required so much passing back and forth of mutexes that the parallelism benefits were washed out by threads waiting to acquire locks on the indices. I think for these benchmarks, simplicity of the solution was key to compete with the super fast LIPP index, especially on the 10% insert workload.